Assignment: Responsive Portfolio Site

(HTML5 + CSS3 Only)

Rules: No JavaScript, no frameworks, no CSS preprocessors, no client-side libraries of any kind. Pure `.html` (HTML5) and `.css` (CSS3) files only. Any `<script>` tag in either will result in a grade of **ZERO**.

Purpose

To ensure you are ready to understand the rest of the semester's work on Full Stack Apps, you need to demonstrate a solid understanding of HTML and CSS.

For a real job assignment, you may be expected to build the front page of the project site you propose to deliver. Your client, who might be a customer, your boss, or colleagues on a larger team, need to have a good idea of what you are building to make sure they agree with your approach. Links on these pages are typically either local links to some section on the page or "dead links" which represent links to other pages in the final deliverable.

Deliverable for this assignment

The front page for a **personal portfolio / resume site** for yourself or a fictional (or real) professional. A full site would have multiple pages, graphics, external links, etc. However, for this assignment only the front page and specific content is required.

> **Note**
> All links on the page that target an anchor on this page or the single required image file must work properly. All *other* links must be set to `https://example.com` or `#` which, if clicked, will bring the user to the standard test domain `example.com` or simpley reload the page, both of which are actions with no effect. None of these other links should be "live."

This front page must contain the following PG-13 acceptable content:

- **"About me" section**
  Describing the person this site is all about. It should include an image that is loaded onto the page from an image file you provide.
- **"Selected projects" section**
  These can be selected publications, movies, videos, etc., each

with a brief description and a link to some other page.

- **"Information section"**
  This should contain an HTML **table** of some information. This could contain entries like Job History, Upcoming Performance Dates, a list of links to other sites with a brief description for each, a sports team schedule including past games with scores and future games, etc.
- **"Contact me" section**
  A simple form into which the user enters some information, such as a (fake) search box with a selector for "match case", a "Contact Me" which gathers the user's name/email and other relevant info, or similar data collection ideas.

  > **Note**
  > We haven't discussed HTML Forms in class. This is another skill you need to develop for Full Stack Dev: discovering how to do stuff on your own. Visit W3Schools' site to get started.

- **"Navigation bar"**
  Located near the top of the page, this *responsive bar* provides links to jump directly to each section. For example, a link like
  `<a href="#about")About</a>`
  that will bring you to
  `<section id="about">`.

Required Deliverables

Your GitHub repository must contain:

- `README.md`
  A description of the full site for which this page is the front page.
- `index.html`
  The front page
- `styles.css`
  A single stylesheet (file) containing *all* CSS directives. There should be no inline `style="..."` attributes in the `HTML` file.
- An `/images` folder containing the **single** image used on the page. It **must** include "Alt text" in case I use a screen reader to "view" your page.
- A `README.md` with a 150–300 word explanation of who the page is about and why you did the HTML5 and CSS3 things the way you did.

Functional Requirements

1. HTML5 structure (required elements)

The page must use, correctly and appropriately (not just present anywhere):

- `<header>`, `<nav>`, `<main>`, `<footer>` primary parts
- At least three `<section>` elements, each with a heading (`<h2>` or lower) and an `id` used as an anchor target
- One `<article>` (e.g., a "featured project" or "blog post" style block) *that uses a different font from the rest of the page*.
- One properly structured `<table>` with `<thead>`, `<tbody>`, and `<th scope="col">` / `<th scope="row">` used correctly
- One `<form>` collecting four different input types (e.g., `text` (single line), `email`, `textarea` (multiple lines), and a `checkbox` or `radio` button). The form **should not** submit anywhere real; using `action="#"` is sufficient.
- Correct use of `<a>` for both internal (in-page anchor) and external navigation
- A logical, single `<h1>` and a heading hierarchy below it that never skips a level. That means skipping from `<h1>` directly to `<h3>` instead of `<h2>` first.

2. Accessibility

- Every `<img>` has meaningful `alt` text (decorative images use `alt=""`)
- Every form `<input>` has an associated `<label>` (explicit `for`/`id` pairing, not just placeholder text - think Screen Readers!)
- The scope specs on the table headers (`<th scope="col">` / `<th scope="row">`) are important for accessibility by screen readers. When a screen reader moves down a column, `scope="col"` prompts the device to read aloud the column title before announcing a cell's data. - from the MDN docs.

You can check for Accessibility issues using the `Lighthouse` tab in the browser Developer Tools. Just display your page, then open the Developer Tools[^1] and click the `Lighthouse` tab.

[^1: On macOS, open the console by pressing fn-F12 or option-cmd-i simultaneously. On Windows or Linux, open the console by pressing Fn-F12 or ctrl-shift-i simultaneously. The console can also be opened via the `View` menu of your browser.]

3. CSS — required techniques

- The CSS style file must contain comments explaining why CSS `Grid` or `FlexBox` are used
- There should be at least one use of `:hover`, `:focus` or similar structural pseudo-class

- The `Article section` should use a different font from the rest of the page.
- No use of the `!important` option to circumvent CSS cascading. **5 points deduction if present**
- No layout done with tables or absolute-positioning hacks **5 points deduction if present**
- Make the page attractive! Potential customers/users won't give your site a chance if it looks boring or hard to use!

4. Responsiveness

- Page must be usable and visually correct at three widths: `375px` (mobile), `768px` (tablet), `1440px` (desktop) — horizontal scrolling, overlapping text, or unreadable font sizes should not occur when the window is resized.
- The image must not overflow its container at any window width.

Explicitly Forbidden

Use of any of the following will result in a **20% point deduction**.

- Any `<script>` tag or `on*=""` inline event handler attributes
- Any JS (JavaScript) framework, bundler, or build step
- Bootstrap, Tailwind, or any other CSS framework. The goal of this assignment is for you to write the CSS yourself
- Copy-pasted boilerplate from page builders — this should visibly be your own work

---

Grading Rubric (20 points)

Category	Points	Criteria
HTML5 structure	4	All required elements present and used correctly per their semantic meaning, not just visual convenience, no inline styling
CSS3 required techniques	4	Grid, Flexbox, media queries, custom properties, pseudo-classes, fluid sizing — all present and marked with comments
Responsiveness	4	Works cleanly at all three tested widths, no overflow/breakage
Appealing design	4	Effective use of color, typography, text, cool techniques like `hover` and `transition`, etc., to make the page attractive
GitHub repository commit history	2	Showing many commits with reasonable commit messages spread over a period of time. Showing proper development Showing proper development hygiene and demonstrating you didn't use AI. and demonstrating you didn't use AI.
Accessibility	2	Alt text, table header scopes, reasonable font sizes, no goofy coloe schemes (like light yellow font on a bright white background).

Automatic penalties:

- Any JavaScript present: **score of 0 for the entire assignment**
- Missing `alt` text on any content image: **−1 per instance**
- Using a table for layout instead of data: **−5**

Suggested Validation Tools

- W3C Markup Validator — HTML correctness
- W3C CSS Validator — CSS correctness
- Browser DevTools' built-in Lighthouse accessibility audit, or the WAVE browser extension — for color choices, alt text checks, table issues, etc.
- Manually dragging the window size to be really narrow or you could learn how to use the Browser DevTools responsive design mode — for the three-width check. Enter this mode by clicking the **Device Toolbar icon** (it looks like a phone and tablet) in the top-left corner of the DevTools panel.

Stretch Goal (optional extra credit, +2)

Implement a link in the footer reading "click here for problems with this page" that will invoke a `mailto` link labeled "Report landing page problem" using this `href`:

```
mailto:c.palmer1@snhu.edu?
subject=Landing%20Page%20Problem&body=Ask%20me%20about%20Landing%20page%20problem!
```

When this link is clicked, the browser will attempt to open the email app configured for the host system and create an email ready to be sent.